Clinical Summary Tool

AI-Powered Workflow System · Track 5: Healthcare · README & Submission Guide

1. Overview

Clinicians spend a lot of time turning long shift notes into handoff summaries. This tool takes a free-text clinical note and returns a **structured, non-diagnostic handoff summary**: a short summary, key points, a **priority level** (Critical / High / Medium / Low), a **risk score** (0–100) with the reasons behind it, and **next steps**. Every result is stored in a database and monitored from an admin dashboard.

AI is one component of a larger pipeline: Claude (or a built-in rule-based summarizer if no API key is set) writes the summary, while a separate, predictable **Smart Logic Layer** decides how urgent the case is.

2. System flow

Stage	What happens	Where in the code
User input	Clinician pastes a note, optional patient ref and author, or picks a sample note	public/index.html
Frontend (UI)	Validates length, sends the note to the API, animates the pipeline strip	public/app.js
Backend API	POST /api/summarize validates input and runs the workflow	server.js
AI generation	Claude returns summary, key points, next steps as JSON; falls back to rule-based if no key or on error	src/ai.js
Smart Logic Layer	Risk keyword detection (with negation handling), vital-sign checks, risk score, priority, action suggestions	src/logic.js
Database	Saves every workflow and an activity log in SQLite	src/db.js → data/clinical.db
Admin dashboard	Stats, priority breakdown, top risks, 7-day usage, activity log, review/delete	public/admin.html, admin.js
Structured output	Priority badge, score bar, risk chips, summary, key points, next steps, copy button	public/index.html

3. Features by layer

Frontend (user screen)

- Note input with character counter, patient reference and author fields
- Four sample notes (critical, high, medium, low) for quick demos
- Live pipeline strip showing Input → API → AI → Logic → Database → Output
- Formatted result card: never raw JSON
- Recent summaries table: click any row to reopen it from the database

Backend + AI

- Express API with input validation (20–10,000 characters)
- Claude prompt that forbids diagnosis, invented facts, and medication advice
- Automatic rule-based fallback, so the app works with or without an API key

Smart Logic Layer

- 14 risk rule groups across critical, high and moderate levels
- Negation handling: “denies chest pain” or “no fever” does not raise risk
- Vital-sign parsing: SpO2, heart rate, blood pressure, temperature (°C or °F)
- Explainable score: each point is traced to a category, shown as “Score built from…”
- Safety actions are listed before AI-written steps and de-duplicated

Admin dashboard

- Protected by an admin key
- Totals, today, open, open critical, average score and processing time, errors
- Priority, top risk category and 7-day usage charts
- Activity log including rejected inputs and AI fallbacks
- Search and filter, view full record, mark reviewed / reopen, delete

4. Smart Logic Layer: how priority is decided

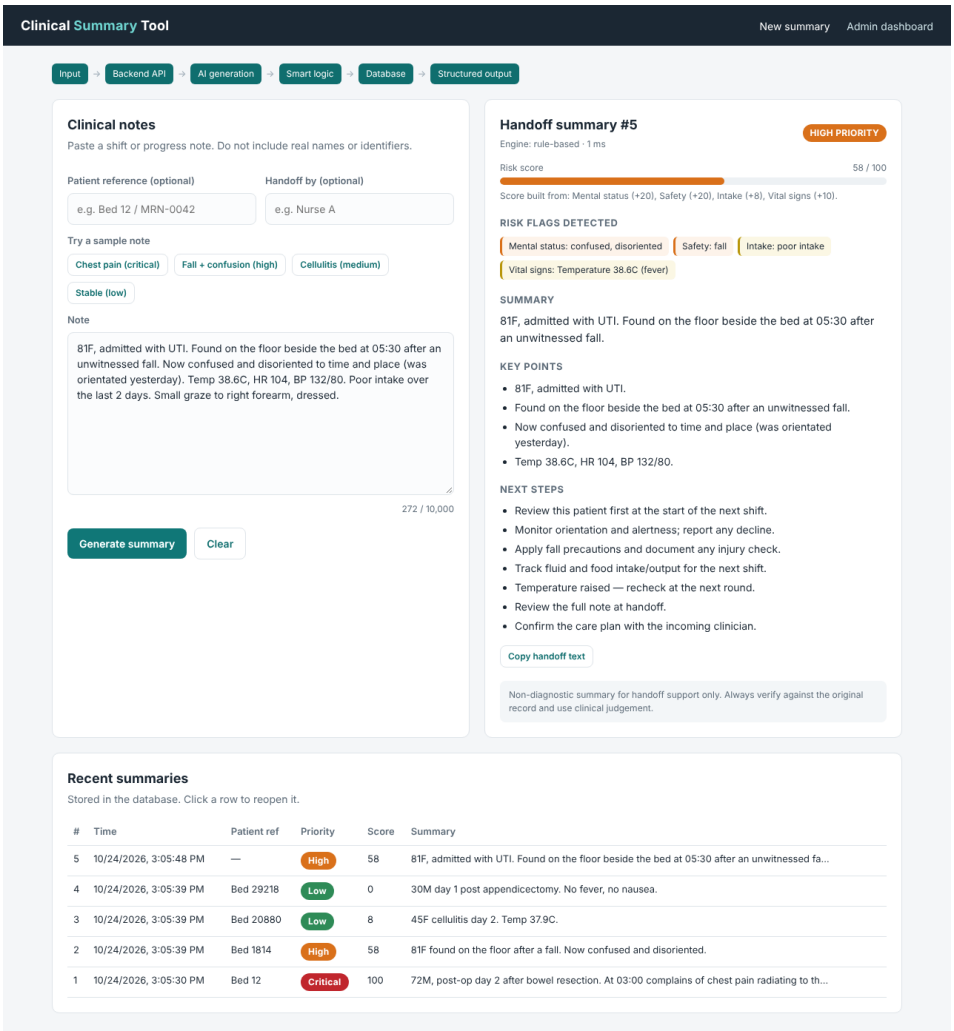
Level	Examples of keywords / readings	Points
Critical	chest pain, shortness of breath, stroke, seizure, unresponsive, suicidal, sepsis, severe bleeding, SpO2 < 90%, HR > 130 or < 40, systolic BP < 90	30–40
High	fever, hypotension, tachycardia, confusion, fall, low oxygen, allergic reaction, SpO2 90–93%, BP ≥ 180, temp ≥ 39.5°C	15–25
Moderate	pain, nausea, dizziness, headache, swelling, missed dose, poor intake, temp 38–39.4°C	8–10

Priority	Rule	Handoff action added
Critical	Any critical flag, or score ≥ 70	Verbal handoff required
High	Score 40–69	Review this patient first next shift
Medium	Score 15–39	Routine handoff, reassess within shift
Low	Score under 15	Continue current care plan

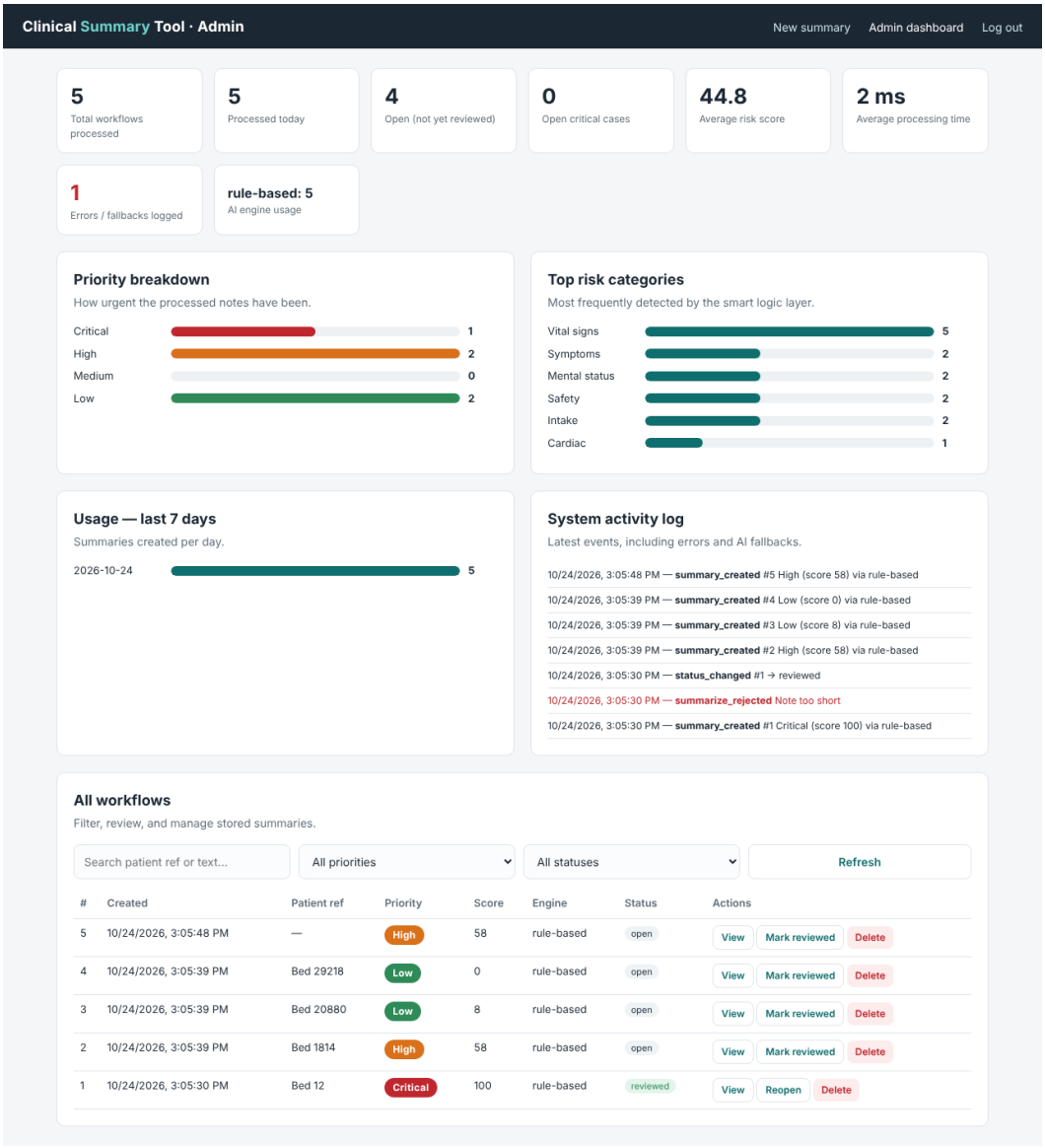
Why rules and not only AI? Priority must be predictable and explainable: the same note always gets the same priority, and every point of the score has a visible reason.

5. Screenshots

User screen: input on the left, structured output on the right, stored history below.



Admin dashboard: monitoring and managing stored workflows.



6. Tech stack and folder structure

Node.js + Express (backend) · plain HTML/CSS/JavaScript (frontend) · SQLite via better-sqlite3 (database) · Claude API (optional AI) · Replit (development) · Git & GitHub (version control).

```
clinical-summary-tool/
├── server.js           Backend API + workflow orchestration
├── src/ai.js           AI generation (Claude + rule-based fallback)
├── src/logic.js        Smart Logic Layer (risk, score, priority, actions)
├── src/db.js           SQLite database layer
├── public/             index.html, app.js, admin.html, admin.js, styles.css
├── test/run-tests.js  Logic layer tests (npm test)
├── data/               clinical.db is created here automatically
├── .env.example · .replit · .gitignore · package.json
└── README.pdf
```

7. API endpoints

Method	Endpoint	Purpose
GET	/api/health	Server status and AI mode
POST	/api/summarize	Main workflow: note in, structured summary out (saved to DB)
GET	/api/summaries	Recent summaries for the user screen
GET	/api/summaries/:id	One stored summary
GET	/api/admin/stats	Dashboard statistics (needs admin key)
GET	/api/admin/summaries	Filter/search all workflows (needs admin key)
PATCH	/api/admin/summaries/:id	Mark reviewed / reopen (needs admin key)
DELETE	/api/admin/summaries/:id	Delete a workflow (needs admin key)

8. Running it on Replit (no coding needed)

1. **Create the Repl.** In Replit, click **Create App** → choose **Import** / upload, and upload the project zip. (Or, once the code is on GitHub, use **Import from GitHub**.)
2. **Add Secrets.** Open the **Secrets** tool (padlock icon) and add **ADMIN_KEY** (your dashboard password). Optionally add **ANTHROPIC_API_KEY** to use Claude. Without it, the rule-based summarizer is used.
3. **Press Run.** Replit installs the packages and starts the server. The web preview opens the user screen.
4. **Open the dashboard** by adding **/admin** to the preview URL and entering your ADMIN_KEY.
5. **Test it.** Click each sample-note button and Generate summary. In the Shell, run **npm test** to run the logic tests (6 should pass).

If Run shows an error about better-sqlite3, open the Shell and type **npm rebuild better-sqlite3**, then press Run again.

9. Git & GitHub: one commit per stage

The project already contains a Git history with one commit per stage, so the development progression is visible as soon as it is pushed to GitHub.

#	Stage	Commit message	Files added
1	Frontend	Stage 1: Frontend UI for clinical note input and structured output	public/index.html, app.js, styles.css, package.json, config files

#	Stage	Commit message	Files added
2	AI generation	Stage 2: AI generation layer (Claude + rule-based fallback)	src/ai.js
3	Smart logic	Stage 3: Smart logic layer (risk keywords, vitals, scoring, priority)	src/logic.js, test/run-tests.js
4	Database	Stage 4: SQLite database layer for workflows and activity log	src/db.js, data/.gitkeep
5	Admin	Stage 5: Admin dashboard for monitoring and managing workflows	public/admin.html, admin.js
6	Integration	Stage 6: Backend API integrating input → AI → logic → DB → output	server.js
7	Docs	Stage 7: README and submission guide (PDF)	README.pdf

Pushing to GitHub from Replit

1. On GitHub, create a new **empty** repository (no README, no .gitignore), e.g. **clinical-summary-tool**.
2. In Replit, open the **Git** tool. It will detect the existing history. Connect your GitHub account when asked.
3. Set the remote to your new repository and click **Push**. All 7 commits appear under **Commits** on GitHub.
4. Any later change: edit, then in the Git tool write a commit message and click **Commit & Push**.

Never commit real API keys. They live only in Replit Secrets; .env is excluded by .gitignore.

10. Submission guidelines

Your submission must include:

- A working full-stack application
- Minimum 6 commits (one per stage)
- Clear frontend ↔ backend ↔ database integration
- Functional admin dashboard
- Structured output (no raw JSON display)
- Visible use of a logic layer

Compliance checklist

Requirement	How this project meets it	How to show it
Working full-stack application	Frontend (public/), Express backend (server.js), SQLite database (src/db.js), all running from one Replit Run	Live Replit URL; generate a summary from a sample note
Minimum 6 commits (one per stage)	7 commits: frontend, AI, logic, database, admin, integration, docs	GitHub → Commits tab
Frontend ↔ backend ↔ database integration	UI calls POST /api/summarize → AI + logic → saved in DB → returned to UI; history table reads back from the DB	Submit a note, then see it appear in Recent summaries and in the admin table
Functional admin dashboard	/admin with stats, charts, activity log, search, filters, view, mark reviewed, delete	Log in at /admin; mark one case reviewed
Structured output (no raw JSON)	Result card with priority badge, risk bar, risk chips, summary, key points, next steps and a copy button	Screenshot of the result card
Visible use of a logic layer	Risk score, priority, “Score built from…” explanation, risk flag chips and rule-based actions on every result; top risk categories on the dashboard	Compare the four sample notes: each lands on a different priority

What to submit

- GitHub repository link (showing at least 6 stage commits)
- Replit app link (running)
- This README.pdf (included in the repository)
- Optional: screenshots or a short screen recording of the user screen and admin dashboard

11. Safety, privacy and limitations

- Non-diagnostic by design: the tool summarises and flags, it does not diagnose or recommend medication.
- Use synthetic or de-identified notes only. Do not paste real patient names or identifiers.
- Keyword rules can miss unusual wording; output must always be checked against the original record.
- The admin key is a simple password suitable for a demo, not production authentication.